Darren Lee
danalee@ucsc.edu
4/18/2021

CSE 13S Spring 2021
Assignment 3: Sorting: Putting your affairs in order
Design Document

Pre-lab Part 1

1. How many rounds of swapping will need to sort the numbers 8,22,7,9,31,5,13 in ascending order using Bubble Sort?

It will take 5 rounds of swapping to put the numbers in ascending order using Bubble Sort. A round of swapping means checking the (n-i) elements starting from i=0 to i=n-2 because you get to check one less element every round. This is because the largest element at the right end of the array no longer needs to be checked. Once all the elements are in order, we are done.
After the first round: 8,7,9,22,5,13,31
After the second round: 7,8,9,5,13,22,31
After the third round: 7,8,5,9,13,22,31
After the fourth round: 7,5,8,9,13,22,31
After the fifth round: 5,7,8,9,13,22,31
Technically, there is a sixth pass in which the numbers are compared again (n=7 and n-1=6 passes), but since the numbers are already in order there is no more swapping after the fifth round.

2. How many comparisons can we expect to see in the worse case scenario for Bubble Sort? Hint:make a list of numbers and attempt to sort them using Bubble Sort.

According to https://www.geeksforgeeks.org/bubble-sort/, Bubble Sort has a worst case time complexity of $O(n^2)$, which occurs when the list is reverse sorted. For example, (5,4,3,2,1) would take 20 comparisons when using Bubble Sort, which is n^2-n for n=5. However, since n^2 is the dominating term and the -n does not make a large difference as the number of terms grows larger, the worst case time complexity is $O(n^2)$.

Pre-lab Part 2

1. The worst time complexity for Shell Sort depends on the sequence of gaps. Investigate why this is the case. How can you improve the time complexity of this sort by changing the gapsize? Cite any sources you used.

Shell Sort's worst time complexity depends on the sequence of gaps because that determines how many comparisons are made. Since we are using the Pratt sequence, the worst time complexity will be $O(n\log^2 n)$, but using the Knuth sequence with the formula $(3^K-1)/2$ would have a worst time complexity of $O(n^{3/2})$. I used https://www.codesdope.com/blog/article/shell-sort/ and https://en.wikipedia.org/wiki/Shellsort#Gap_sequences to answer this question.

Pre-lab Part 3

1. Quicksort, with a worst case time complexity of $O(n^2)$, doesn't seem to live up to its name. Investigate and explain why Quicksort isn't doomed by its worst case scenario. Make sure to cite any sources you use.

The worst cases of Quick Sort occur when the array is sorted from least to greatest and the leftmost element is chosen as the pivot and when the array is sorted from greatest to least and the rightmost element is chosen. The array will be split into two very unbalanced sub-arrays, one with length 1 and one with length n-1. There are ways to avoid these scenarios such as picking a pivot from the middle of the array, selecting random pivots, or taking the median of 3 pivots. On average, the time complexity of Quick Sort is O(nlogn). It performs well even with large arrays and is considered one of the best sorts regarding efficiency. I used https://www.baeldung.com/cs/quicksort-time-complexity-worst-case to answer this question.

Pre-lab Part 4

1. Explain how you plan on keeping track of the number of moves and comparisons since each sort will reside within its own file.

I plan to use global variables in my sorting header files to keep track of the number of moves and comparisons for Bubble Sort and Shell Sort. For each comparison, I can increment the variable that counts comparisons, and for each movement in the array, I can increment the variable that counts moves. For Quick Sort, I can create helper functions that are placed in the while loops for the Quick Sort algorithm that increment a global variable in the Quick Sort header file whenever a comparison is made. Moves for Quick Sort will be counted just like Bubble Sort and Shell Sort using a global variable that is incremented.

Sources:

1. https://www.geeksforgeeks.org/bubble-sort/
2. https://www.codesdope.com/blog/article/shell-sort/
3. https://en.wikipedia.org/wiki/Shellsort#Gap_sequences
4. https://www.baeldung.com/cs/quicksort-time-complexity-worst-case

---

Purpose

The purpose of this program is to implement Bubble Sort, Shell Sort, and two Quick Sorts. The program uses abstract data types, a stack for the first Quick Sort and a queue for the second Quick Sort. This lab will allow me to compare the sorting algorithms and better understand their implementations. I will accomplish this by collecting statistics on the number of sorts and comparisons for each sort. I will also keep track of the maximum stack size and maximum queue size for the two iterative Quick Sorts. I will also implement a few different abstract data types with the stack and queue for the Quick Sorts and a set for the command line arguments in the test harness. The test harness will test the sorts and print out the collected statistics.

Layout/Structure

- The header files bubble.h, shell.h, and quick.h have the interfaces and function prototypes for the sorts. They also contain the global variables that collect numbers of moves and comparisons for the sorts.
- Sorting.c contains a test harness for the sorting algorithms which creates an array of random elements to be sorted. It takes the command line arguments [-absqQ] [-n length] [-p elements] [-r seed].
    - -a option runs all sorts
    - -b option runs Bubble Sort
    - -s options runs Shell Sort
    - -q option runs Quick Sort that uses stack
    - -Q option runs Quick Sort that uses queue
    - -n length option specifies the number of elements in the array
    - -p elements option specifies the number of elements to print
    - -r seed option specifies the random seed for the pseudorandom numbers in the array
- Bubble.c implements bubble sort, shell.c implements shell sort, quick.c implements both iterative Quick Sorts, stack.c implements the stack abstract data type, and queue.c implements the queue abstract data type.
- Quick.c also has the global variables for the maximum queue size and maximum stack size. Furthermore, it has two helper functions that help count the number of comparisons for the two Quick Sorts.
- Set.c implements the set abstract data type. Set.h contains the function prototypes for set.c.

Functionality

- Bubble sort swaps a pair of items in an array if one is smaller than the other. The largest element reaches the top of the array. The time complexity in the worst case is $O(n^2)$.
- Shell sort pairs up elements that have a large gap between them. Using gaps in a gap sequence, it swaps pairs if they are in the wrong order and starts with distant elements. The expected time complexity depends on the gap sequence. I will use the Pratt sequence (2^p3^q).
- Quick Sort partitions arrays into two sub-arrays by choosing an element from the array to be a pivot. Elements less than the pivot are placed in the left sub-array, and elements that are greater than or equal to the pivot are placed in the right sub-array. It has a worst case time complexity of $O(n^2)$.
- The two Quick Sorts implemented in this program are iterative. One uses a stack and the other uses a queue.
- The test harness will support the following options:
    - -a: runs all sorting algorithms
    - -b: runs bubble sort
    - -s: runs shell sort
    - -q: runs quicksort that uses stack
    - -Q: runs quicksort that uses queue

    - -r seed: sets random seed; default seed is 13371453
    - -n size: sets the array size; default size is 100
    - -p elements: prints out *elements* number of elements from array; default number of elements to print out is 100

- If the user enters an array size of 0 or a negative array size, the sorting program will print out an error message. If an invalid argument is entered or no arguments are entered in the command line, the program will print a usage guide. If no sorts are specified, the program will print a message saying to choose at least one sort to perform. I did not implement error handling for negative seeds or negative numbers of elements to print.
- The test harness will create a random array using the provided seed. The same array will be created for each sorting algorithm by seeding srandom() again with the same seed for each sort. The command line arguments for the test harness are kept track of using a set abstract data type.
- Moves for Bubble Sort, Shell Sort, and Quick Sort are counted using global variables in the header files of the sorting algorithms. Comparisons for Bubble Sort and Shell Sort are counted using global variables in the header files as well. For Quick Sort, the comparisons are counted using helper functions that are called in the comparison while loops. These functions make a comparison and increment the comparison counter global variable for Quick Sort. The maximum stack and queue sizes are tracked using global variables in the quick.c file.

Pseudocode

set.c

Set abstract data type code was provided by Eugene on Piazza

```
set_empty():
        Return 0x00000000

set _member(Set s, x):
        Return s & (1 << (x % SET_CAPACITY)

set_insert(Set s, x):
        Return s | (1<<(x % SET_CAPACITY)

set_remove(Set s, x):
        Return s & ~(1<<(x % SET_CAPACITY))

Set_union(Set s, x):
        Return s | t

set_intersect(Set s, Set t):
        Return s & t
```

```
set_difference(Set s, Set t):
        Return s & ~t

set_complement(Set s):
        Return ~s
```

Python pseudocode was provided by Professor Long in assignment pdf

bubble.c

```
bubble_sort(*A,n):
        Declare int temp
        swapped=1
        While (swapped):
                swapped=0
                For (i=1; i<n; i++):
                        B_compares++
                        If (A[i]<A[i-1]):
                                temp=A[i]
                                A[i]=A[i-1]
                                A[i-1]=temp
                                swapped=1
                                b_moves+=3
                n-=1
```

Python pseudocode was provided by Professor Long in assignment pdf

shell.c

```
shell_sort(*A, n):
        Declare int j
        Declare int temp
        For (k=0;k<sizeof(gaps)/4; k++):
                j=i
                temp=A[i]
                s_moves++
                While (j>=gaps[k] and temp<A[j-gaps[k]]
                        s_compares++
                        A[j]=A[j-gaps[k]]
                        s_moves++
                        j-=gaps[k]
                If (j>=gaps[k]):
                        s_compares++
                A[j]=temp
                s_moves++
```

quick.c

```
Q_less_than_counter(*A, x, p):
        Q_compares++
        Return A[x]<p

Q_greater_than_counter(*A, x, p):
        Q_compares++
        Return A[x]>p
```

Python pseudocode was provided by Professor Long in assignment pdf

```
partition(*A, lo, hi):
        pivot=A[log+(hi-lo)/2]
        i=lo-1
        j=hi+1
        Int temp
        Do:
                I++
                while(q_less_than_counter(A,i,pivot):
                        I++
                while(q_greater_than_counter(A,j,pivot):
                        J--
                If (i<j):
                        temp=A[i]
                        A[i]=A[j]
                        A[j]=temp
                        q_moves+=3
        while(i<j)
        Return j

quick_sort_stack(*A, n):
        lo=0
        hi=n+1
        *p_hi=&hi
        *p_lo=&lo
        Stack *s=stack_create(2*n)
        stack_push(s,lo)
        stack_push(s,hi)
        While (!stack_empty(s))
                stack_pop(s,p_hi)
                stack_pop(s,p_lo)
                p=partition(A,lo,hi)
                if(lo<p):
                        stack_push(s,lo)
                        stack_push(s,p)
```

```
            if(hi>p+1):
                stack_push(s,p+1)
                stack_push(s,hi)
            if(stack_size(s)>max_stack_size)
                max_stack_size=stack_size(s)
            stack_delete(&s)

quick_sort_queue(*A,n):
    lo=0
    hi=n+1
    *p_hi=&hi
    *p_lo=&lo
    Queue *q=queue_create(2*n)
    enqueue(q,lo)
    enqueue(q,hi)
    While (!queue_empty(q))
        dequeue(q,p_hi)
        dequeue(q,p_lo)
        p=partition(A,lo,hi)
        if(lo<p):
            enqueue(q,lo)
            enqueue(q,p)
        if(hi>p+1):
            enqueue(q,p+1)
            enqueue(q,hi)
        if(queue_size(q)>max_queue_size)
            max_queue_size=queue_size(q)
        queue_delete(&q)
```

stack.c

Code provided by Professor Long in assignment pdf and lecture 11

Struct Stack { top,capacity,*items }

```
Stack *stack_create(capacity):
    Stack *s=(Stack *) malloc(sizeof(Stack))
    If (s):
        s->top=0
        s->capacity=capacity
        s->items=(int *) calloc(capacity,sizeof(int))
        if(!s->items):
            free(s)
            s=null
    Return s
```

```
stack_delete(Stack **s):
        if(*s && (*s)->items):
                free((*s)->items)
                free (*s)
                *s=null

stack_empty(Stack *s):
        Return !(s->top)

stack_full(Stack *s):
        s->top==s->capacity

stack_size(Stack *s):
        Return s->top
```

Code influenced by Eugene's 4/22 lab section

```
stack_push(Stack *s, x)
        if(stack_full(s)):
                Return false
        Else:
                (s->items)[s->top]=x
                s->top++
                Return true

stack_pop(Stack *s, *x)
        if(stack_empty(s)):
                Return false
        Else:
                s->top--
                *x=s->items[s->top]
                Return true

stack_print(Stack *s):
        For (i=0; i<s->top; i++):
                print(s->items[i])
        print("\n")
```

queue.c

Struct code provided by Professor Long in assignment pdf

```
Struct Queue { head, tail,size,capacity,*items }
```

code influenced by stack constructor code provided by Professor Long in assignment pdf

```
Queue *queue_create(capacity):
        Queue *q=(Queue *) malloc(sizeof(Queue))
```

```
	If (q):
		q->head=0
		q->tail=0
		q->capacity=capacity
		q->items=(int *) calloc(capacity,sizeof(int))
		if(!q->items):
			free(q)
			q=null
	Return q
```

code influenced by stack destructor code provided by Professor Long in assignment pdf

```
queue_delete(Queue **q):
	if(*q && (*q)->items):
		free((*q)->items)
		free (*q)
		*q=null

queue_empty(Queue *q):
	Return !q->size

queue_full(Queue *q):
	Return q->size==q->capacity

queue_size(Queue *q):
	Return q->size;
```

influenced by Eugene's 4/22 lab section

```
enqueue(Queue *q, x):
	if(queue_full(q):
		Return false
	Else:
		q->items[q->tail]=x
		q->tail=(q->tail+1)%q->capacity
		q->size++
		Return true

dequeue(Queue *q, *x):
	If *(queue_empty(q)):
		Return false
	Else:
		*x=q->items[q->head]
		q->head=(q->head+1)%q->capacity
		q->size--
		Return true
```

```
queue_print(Queue *q):
	For (int i=q->head;i<q->tail; i++):
		print(q->items[i])
	Print (“\n”)
```

sorting.c

```
print_bubble( *A, n, p) {
	b_moves = b_compares = 0
	bubble_sort(A, n)
	print("Bubble Sort\n")
	// code for formatting print statement provided by Professor Long in assignment pdf
	print(
	"%" PRIu32 " elements, %" PRIu32 " moves, %" PRIu32 " compares\n", n, b_moves,
b_compares)
	i = 0
	while (i < p && i < n):
		for (int j = 0; j < 5; j++):
			print("%13" PRIu32, A[i])
			i++
			if (i >= p || i >= n):
				break
		print("\n")

print_shell( *A, n, p) {
	s_moves = s_compares = 0
	shell_sort(A, n)
	print("Shell Sort\n")
	// code for formatting print statement provided by Professor Long in assignment pdf
	print(
	"%" PRIu32 " elements, %" PRIu32 " moves, %" PRIu32 " compares\n", n, s_moves,
s_compares)
	i = 0
	while (i < p && i < n):
		for (int j = 0; j < 5; j++):
			print("%13" PRIu32, A[i])
			i++
			if (i >= p || i >= n):
				break
		print("\n")

print_quick_stack( *A, n, p) {
	q_moves = q_compares = 0
	quick_sort_stack(A, n)
```

```
	print("Quick Sort (Stack)\n")
	// code for formatting print statement provided by Professor Long in assignment pdf
	print(
	"%" PRIu32 " elements, %" PRIu32 " moves, %" PRIu32 " compares\n", n, q_moves,
q_compares)
	print("Max stack size: %" PRIu32 "\n", max_stack_size)
	i = 0
	while (i < p && i < n):
		for (int j = 0; j < 5; j++):
			print("%13" PRIu32, A[i])
			i++
			if (i >= p || i >= n):
				break
		print("\n")

print_quick_queue( *A, n, p) {
	q_moves = q_compares = 0
	quick_sort_queue(A, n)
	print("Quick Sort (Queue)\n")
	// code for formatting print statement provided by Professor Long in assignment pdf
	print(
	"%" PRIu32 " elements, %" PRIu32 " moves, %" PRIu32 " compares\n", n, q_moves,
q_compares)
	print("Max queue size: %" PRIu32 "\n", max_queue_size)
	i = 0
	while (i < p && i < n):
		for (int j = 0; j < 5; j++):
			print("%13" PRIu32, A[i])
			i++
			if (i >= p || i >= n):
				break
		print("\n")

main(argc,**argv):
	code inspired by Eugene's lab section on 4/22
	(*sort_functions[4])(* A, n, p) = { print_bubble, print_shell, print_quick_stack,
print_quick_queue }
	typedef enum opts { b, s, q, Q } opts
	Set opt_set = set_empty()
	opt = 0
	seed = 13371453
	size = 100, elem = 100
	while ((opt = getopt(argc, argv, OPTIONS)) != -1):
		switch (opt):
```

```
case 'a':
        opt_set = set_insert(opt_set, b)
        opt_set = set_insert(opt_set, s)
        opt_set = set_insert(opt_set, q)
        opt_set = set_insert(opt_set, Q)
break
case 'b':
        opt_set = set_insert(opt_set, b)
        break
case 's':
        opt_set = set_insert(opt_set, s)
        break
case 'q':
        opt_set = set_insert(opt_set, q)
        break
case 'Q': opt_set = set_insert(opt_set, Q)
        break
case 'r':
        seed = strtoul(optarg, NULL, 10)
        break
case 'n':
        size = strtoul(optarg, NULL, 10)
        if (size == 0):
                fprintf(stderr, "Invalid array length.\n")
        return 1
        if (size < 0):
                fprintf(stderr, "Failed to allocate array to sort.\n")
                return 1
        break
case 'p':
        elem = strtoul(optarg, NULL, 10)
        Break
default:
        printf("SYNOPSIS\n")
        printf(" A collection of comparison-based sorting algorithms.\n")
        printf("\n")
        printf("USAGE\n   ./ sorting [-absqQ] [-n length] [-p elements] [-r seed]\n")
        printf("\n")
        printf("OPTIONS\n")
        printf("   -a              Enable all sorts.\n")
        printf("   -b              Enable Bubble Sort.\n")
        printf("   -s              Enable Shell Sort.\n")
        printf("   -q              Enable Quick Sort(Stack).\n")
        printf("   -Q              Enable Quick Sort(Queue).\n")
```

```
                printf("   -n length       Specify number of array elements.\n")
                printf("   -p elements     Specify number of elements to print.\n")
                printf("   -r seed         Specify random seed.\n")
                return 1
Arr[size]
for (opts i = b; i <= Q; i++):
        if (set_member(opt_set, i)):
                srandom(seed);
                for (int i = 0; i < size; i++):
                        arr[i] = random()
                sort_functions[i](arr, size, elem)
if (!set_member(opt_set, b) && !set_member(opt_set, s) && !set_member(opt_set, q)
        && !set_member(opt_set, Q)):
        printf("Select at least one sort to perform.\n")
        printf("SYNOPSIS\n")
        printf(" A collection of comparison-based sorting algorithms.\n")
        printf("\n")
        printf("USAGE\n   ./ sorting [-absqQ] [-n length] [-p elements] [-r seed]\n")
        printf("\n")
        printf("OPTIONS\n")
        printf("   -a              Enable all sorts.\n")
        printf("   -b              Enable Bubble Sort.\n")
        printf("   -s              Enable Shell Sort.\n")
        printf("   -q              Enable Quick Sort(Stack).\n")
        printf("   -Q              Enable Quick Sort(Queue).\n")
        printf("   -n length       Specify number of array elements.\n")
        printf("   -p elements     Specify number of elements to print.\n")
        printf("   -r seed         Specify random seed.\n")
        return 1

return 0
```

Paper Work

Bubble Sort

	8, 7, 9, 22, 5, 13, 31	$n = 6$
	7, 8, 9, 5, 13, 22, 31	5
	7, 8, 5, 9, 13, 22, 31	4
	7, 5, 8, 9, 13, 22, 31	3
	5, 7, 8, 9, 13, 22, 31	2
done	5, 7, 8, 9, 13, 22, 31	1

Shell Sort

		Gap
[9, 4, 1, 5, 6, 3]	temp=6	4
[6, 4, 1, 5, 9, 3]	temp=3	2
[6, 3, 1, 5, 9, 4]	temp=1	1
[1, 3, 6, 5, 9, 4]	temp=9	
[1, 3, 6, 5, 9, 4]	temp=5	
[1, 3, 6, 5, 9, 4]	temp=4	
[1, 3, 6, 4, 9, 5]	temp=3	
[1, 3, 6, 4, 9, 5]	temp=6	
[1, 3, 6, 4, 9, 5]	temp=4	
[1, 3, 4, 6, 9, 5]	temp=9	
[1, 3, 4, 6, 9, 5]	temp=5	
[1, 3, 4, 6, 5, 9]	temp=5	
[1, 3, 4, 5, 6, 9]	done	

Quick Sort

pivot

[1 9 2 3 4 5 7]

pivot

[3 1 9 2 4 5 7]

i, > pivot

pivot

[3 1 9 2 4 5 7]

j

< pivot

p

[3 1 2 9 4 5 7]

p i

[3 1 2 9 4 5 7]

[[1 2] 3 [9 4 5 7]]

p j

[1 2 3 [7 4 5] 9]

p j

[1 2 3 [5 4] 7 9]

[1 2 3 4 5 7 9]